

Big O Doesn't Matter When n Is Small, and n Is Usually Small

Armed with the knowledge of big O notation, you might be eager to analyze every piece of code you write. Before you start using this tool to hammer every nail in sight, keep in mind that big O notation is most useful when there is a large amount of data to process. In real-world cases, the amount of data is usually small.

In those situations, coming up with elaborate, sophisticated algorithms with lower big O orders might not be worth the effort. Go programming language designer Rob Pike has five rules about programming, one of which is: “Fancy algorithms are slow when ‘ n ’ is small, and ‘ n ’ is usually small.” Most software developers won’t be dealing with massive data centers or complex calculations but rather more mundane programs. In these circumstances, running your code under a profiler will yield more concrete information about the code’s performance than big O analysis.

Summary

The Python standard library comes with two modules for profiling: `timeit` and `cProfile`. The `timeit.timeit()` function is useful for running small snippets of code to compare the speed difference between them. The `cProfile.run()` function compiles a detailed report on larger functions and can point out any bottlenecks.

It’s important to measure the performance of your code rather than make assumptions about it. Clever tricks to speed up your program might actually slow it down. Or you might spend lots of time optimizing what turns out to be an insignificant aspect of your program. Amdahl’s Law captures this mathematically. The formula describes how a speed-up to one component affects the speed-up of the overall program.

Big O is the most widely used practical concept in computer science for programmers. It requires a bit of math to understand, but the underlying concept of figuring out how code slows as data grows can describe algorithms without requiring significant number crunching.